my eyes, primarily because you must configure the `HttpClient` to point to your service before you use it. If you need to create an `HttpClient` to call the API in more than one place in your application, you must configure it in more than one place too.

`IHttpClientFactory` provides a convenient solution to this problem by allowing you to centrally configure *named clients*, which have a `string` name and a configuration function that runs whenever an instance of the named client is requested. You can define multiple configuration functions that run in sequence to configure your new `HttpClient`.

The following listing shows how to register a named client called `"rates"`. This client is configured with the correct `BaseAddress` and sets default headers that are to be sent with each outbound request. Once you have configured this named client, you can create it from an `IHttpClientFactory` instance using the name of the client, `"rates"`.

Listing 33.4 Using `IHttpClientFactory` to create a named `HttpClient`

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
builder.Services.AddHttpClient("rates", (HttpClient client) =>   ❶
{
    client.BaseAddress =                                         ❷
        new Uri("https://example.com/rates/");                   ❷
    client.DefaultRequestHeaders.Add(                            ❷
        HeaderNames.UserAgent, "ExchangeRateViewer");            ❷
})
.ConfigureHttpClient((HttpClient client) => {})                  ❸
.ConfigureHttpClient(
    (IServiceProvider provider, HttpClient client) => {});       ❹

WebApplication app = builder.Build();

app.MapGet("/", async (IHttpClientFactory factory) =>            ❺
{
```